



EMERITUS

Learn. From the world's best.

Support Vector Machines: Beyond the Basics



Agenda

Understanding SVM: Comparison, Challenges and Applications

SVM vs Other Models

Compare SVM with logistic regression, decision trees, random forests and neural networks

Scaling and Limitations of SVM

Computational complexity, memory use, and handling big data

Industry and Research Trends

How SVM is used in industries and new research directions

Hands-on Case: Spam Detection

Build and test an email classifier using SVM

SVM vs. Logistic Regression

Key Differences in Approach and Use

Logistic Regression

- Works best with clearly separable data
- Gives probability scores for predictions
- Easy to understand and explain
- Fast and handles large data well

VS

Support Vector Machine

- Aims to separate classes with the widest margin
- Uses only key boundary data points
- Performs well in high-dimensional spaces
- Handles complex patterns using kernel trick

SVM vs Decision Trees and Random Forests

Comparing Structure, Scalability and Overfitting

1

Decision Trees

- Handle categories directly, no need for extra processing
- Easy to read with clear decision rules
- Prone to overfitting with deep trees
- No need for scaling, handles outliers well

2

Random Forests

- Combines trees to reduce overfitting
- Works well with missing values using default settings
- Shows feature importance easily
- Trains faster with parallel processing

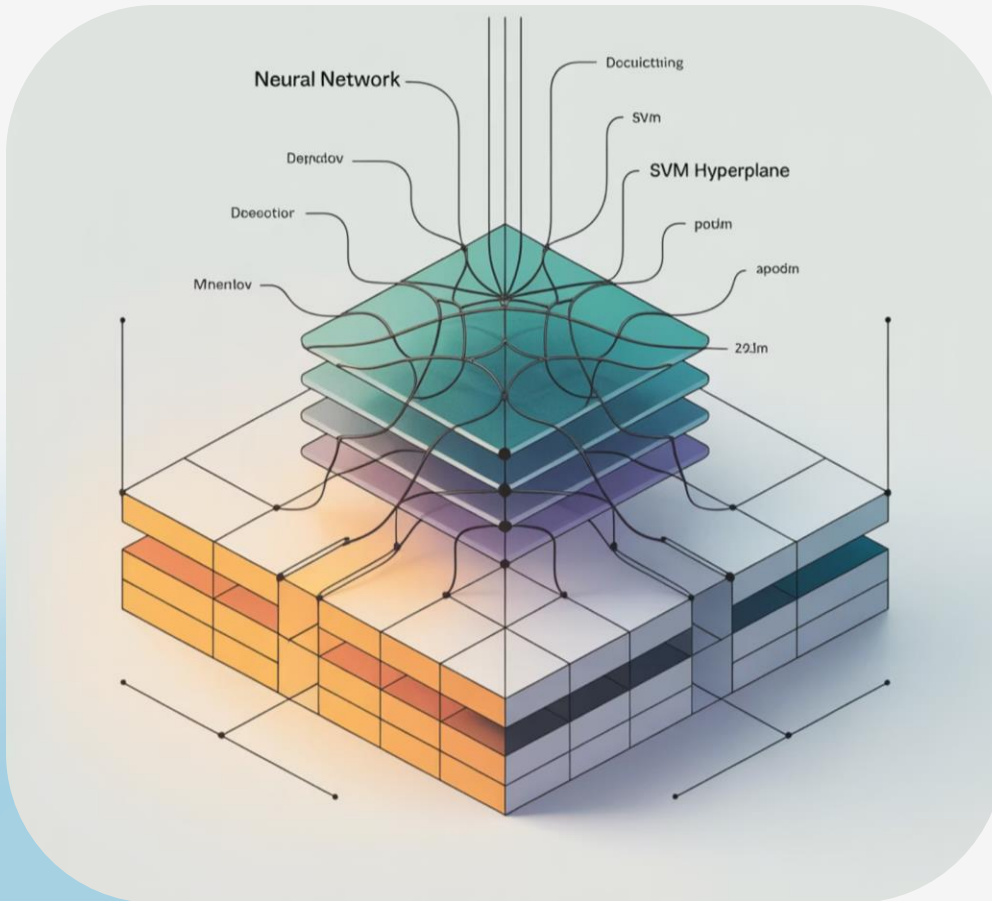
3

Support Vector Machines

- Needs scaled features to work well
- Works best with high-dimensional, small-to-medium data
- Memory-intensive due to kernel matrix computation
- Less likely to overfit in complex spaces

SVM vs Neural Networks

Flexibility, Complexity and Use Cases



Neural Networks

- Very flexible, good for complex patterns
- Needs lots of data and processing power
- Hard to understand (black box)
- Finds features on its own in deep models

Support Vector Machines

- Easier to tune with fewer settings
- Slower with very large data sets
- Works well as a starting model for text and image tasks
- Finds the best solution reliably

When Should You Use SVM?

Best-Fit Scenarios vs When to Avoid It



Use SVM When:

- Dataset is small to medium (<10K samples)
- Features are high-dimensional (like text, genetics)
- Classes are clearly separable
- Require a theoretically well-founded approach



Avoid SVM When:

- Dataset is very large (millions of samples)
- Data is noisy or classes overlap heavily
- Needing well-calibrated probability outputs
- Prioritising fast training or retraining

Computational Complexity

Training Time and Memory Challenges

- SVM scales between $O(n^2)$ and $O(n^3)$, where n is the number of training samples
 - Needs lots of memory to store the kernel matrix (quadratic space)
 - Worst-case scenarios may require cubic time
 - Memory usage rises quickly with larger datasets
- Memory and time needs grow sharply with dataset size, limiting large-scale use

Scaling SVMs

Efficient Techniques for Handling Large Datasets with SVMs



LinearSVC

- Uses LIBLINEAR for fast linear SVMs
- Scales with number of samples
- Best for high-dimensional sparse data



SGDClassifier (Hinge Loss)

- Uses stochastic gradient descent
- Ideal for very large datasets
- Supports online learning with fixed memory



Approximate Kernel Methods

- Uses Random Fourier Features and Nystroem
- Lowers kernel computation cost
- Makes non-linear SVMs scalable

Practical Limitations of SVMs

Challenges Faced when Applying SVMs to Real-World Data

1

Performance Degradation

Training time grows quickly with dataset size (>100K samples)

2

Feature Scaling Sensitivity

Very sensitive to input scales; normalisation is essential

3

Interpretability Challenges

Kernel transformations make results difficult to understand

4

Multi-class Limitations

Needs extra strategies for multi-class tasks, adding time and complexity

When Not to Use SVM

Situations Where SVMs May Not be The Best Choice



Big Data Issues

- Slow and memory-heavy with millions of rows ($O(n^2)$ scaling)

Too Many Irrelevant Features

- Unimportant data weakens performance if not filtered

Needs Clear Explanations

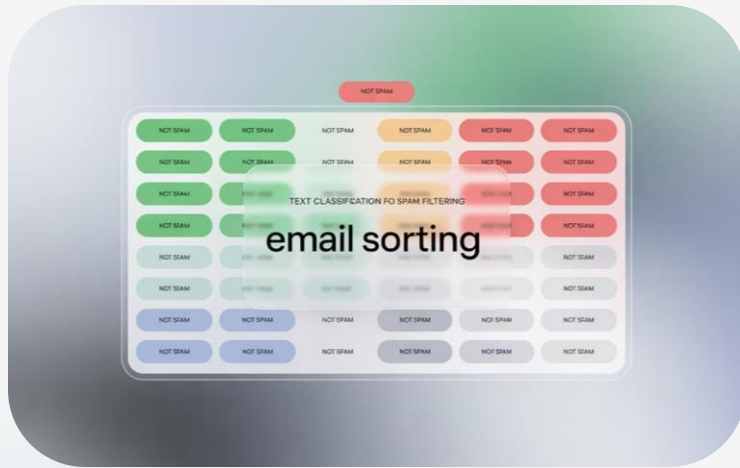
- Hard to explain results due to complex decision boundaries

Frequent Model Updates

- Not ideal for systems needing regular retraining

Industry Applications of SVMs

Where SVMs are Effectively used Across Sectors



Text Classification

- Used for spam filtering, sentiment analysis, and document tagging
- Handles high-dimensional text data well



Bioinformatics

- Applied in gene and protein analysis, and cancer type detection
- Works well with few samples and many features



Anomaly Detection

- Helps in fraud detection, quality checks, and spotting network threats
- One-class SVMs are used to find outliers

SVM in Recent Research

New Developments to Enhance SVM Performance and Flexibility

1

Scalable Kernel Methods

Approximate methods like Nyström and Fourier features boost SVM scalability

2

SVM + Deep Learning Hybrids

Combines deep learning features with SVM's strong theoretical backing

3

Imbalanced Data Handling

Cost-sensitive SVMs and sampling tackle skewed class distributions

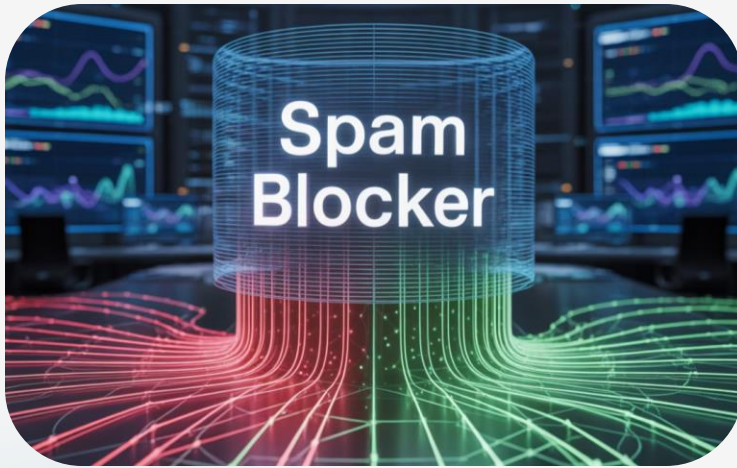
4

Hyperparameter Automation

Bayesian and meta-learning help fine-tune SVM settings more easily

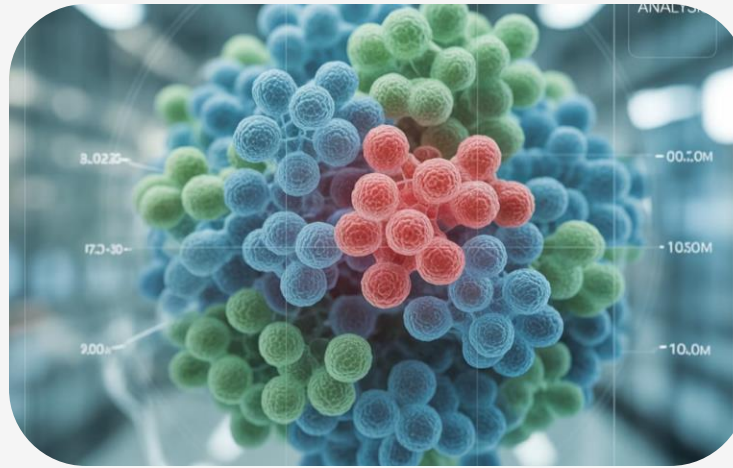
Real-World Case Examples

How SVMs are Used Effectively in Real Applications



Email Spam Filtering

- Used by major providers with >99% accuracy and low false positives
- Linear SVMs retrained regularly for better results



Cancer Diagnosis

- Helps classify cancer types from gene data
- Supports personalised treatment with small sample sizes



Manufacturing QC

- Detects defects in semiconductor production with 99.8% precision
- Reduces the need for manual checks using image-based SVMs

Spam Detection with SVM – Why and How?

SVM for Spam Filtering: Why and How

Why Use SVM for Spam?

- Handles large, high-dimensional text data efficiently
- Ignores irrelevant email features effectively
- Linear kernel balances performance and simplicity
- Focuses on key boundary cases for better accuracy

Implementation Steps

1. Preprocess emails (tokenise, normalise, remove stopwords)
2. Extract features using TF-IDF
3. Split into training and testing sets
4. Train linear SVM with suitable C value
5. Evaluate using precision, recall, and F1 score

Data Preparation for Spam Detection

Steps to Clean and Convert Raw Emails into Features for SVM



Raw Email Data

- Contains labelled emails: “spam” or “ham”
- Common sources: Enron-Spam, SpamAssassin, Ling-Spam



Text Preprocessing

- Convert text to lowercase
- Remove punctuation and symbols
- Remove stopwords (e.g. *the, and, or*)
- Apply stemming or lemmatisation



Feature Extraction

- Tokenise text into words
- Build document-term matrix
- Use TF-IDF to weigh terms
- Optionally use n-grams for better context

TF-IDF Vectorization

TF-IDF Vectorisation

Finding Important Words in Text

- TF-IDF stands for Term Frequency–Inverse Document Frequency
- Measures how important a word is in a document
- Importance is based on how often the word appears in one document vs the whole collection
- Helps highlight unique or meaningful words in a text

TF-IDF: Key Concepts

Understanding Word Frequency and Importance

Term Frequency (TF)

Shows how often a word appears in a document

Formula:

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF)

Shows how unique or rare a word is across documents

Formula:

$$\text{IDF}(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t} \right)$$

TF-IDF Score:

Combines TF and IDF to determine the importance of a term in a document:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

TF-IDF Applications

Where and How It's Used

Text Classification

Helps turn text into features for categorising documents

Information Retrieval

Improves search results by ranking useful documents higher

Advantages and Limitations

What It Does Well and Where It Falls Short

Advantages

- **Fewer Features:** Focuses only on important words
- **Good for Sparse Data:** Works well with text that has lots of empty spaces

Limitations

- **No Context:** Doesn't understand word meaning or usage
- **Fixed Word List:** Can't handle new words not seen during training

TF-IDF Example

Step-by-Step for a Small Dataset

Step 1: Dataset

Use these three sample documents:

1. Doc 1: *"The cat sat on the mat."*
2. Doc 2: *"The dog sat on the log."*
3. Doc 3: *"The cat chased the dog."*

TF-IDF Example – Step 2

Preprocessing the Text

- 1. Tokenise:** Break each sentence into words
- 2. Lowercase:** Convert all words to small letters
- 3. Remove Stopwords:** Take out common words like *the, on, sat*

Processed Text:

- Doc 1: [cat, sat, mat]
- Doc 2: [dog, log]
- Doc 3: [cat, chased, dog]

TF-IDF Example – Step 3

Calculating Term Frequency (TF)

- Count how often each word appears in a document
- Divide by the total number of words in that document

Term	Doc 1 TF	Doc 2 TF	Doc 3 TF
cat	$1/3 = 0.33$	$0/3 = 0.00$	$1/3 = 0.33$
dog	$0/3 = 0.00$	$1/3 = 0.33$	$1/3 = 0.33$
sat	$1/3 = 0.33$	$0/3 = 0.00$	$0/3 = 0.00$
mat	$1/3 = 0.33$	$0/3 = 0.00$	$0/3 = 0.00$
log	$0/3 = 0.00$	$1/3 = 0.33$	$0/3 = 0.00$
chased	$0/3 = 0.00$	$0/3 = 0.00$	$1/3 = 0.33$

TF-IDF Example – Step 4

Inverse Document Frequency (IDF)

Find out how rare a word is across all documents

Formula:

$$\text{IDF}(t) = \log \left(\frac{N}{n_t} \right)$$

N = total number of documents

n_t = number of documents with the term t

TF-IDF Example

Calculation

Total documents $N=3$

Term	Documents Containing Term	IDF
cat	2	$\log(3/2) \approx 0.176$
dog	2	$\log(3/2) \approx 0.176$
sat	2	$\log(3/2) \approx 0.176$
mat	1	$\log(3/1) \approx 0.477$
log	1	$\log(3/1) \approx 0.477$
chased	1	$\log(3/1) \approx 0.477$

TF-IDF Example – Step 5

Final Step: Multiply TF and IDF

Multiply the TF and IDF values for each term in each document

Term	Doc 1 TF-IDF	Doc 2 TF-IDF	Doc 3 TF-IDF
cat	$0.33 \times 0.176 \approx 0.058$	$0.00 \times 0.176 = 0.00$	$0.33 \times 0.176 \approx 0.058$
dog	$0.00 \times 0.176 = 0.00$	$0.33 \times 0.176 \approx 0.058$	$0.33 \times 0.176 \approx 0.058$
sat	$0.33 \times 0.176 \approx 0.058$	$0.00 \times 0.176 = 0.00$	$0.00 \times 0.176 = 0.00$
mat	$0.33 \times 0.477 \approx 0.157$	$0.00 \times 0.477 = 0.00$	$0.00 \times 0.477 = 0.00$
log	$0.00 \times 0.477 = 0.00$	$0.33 \times 0.477 \approx 0.157$	$0.00 \times 0.477 = 0.00$
chased	$0.00 \times 0.477 = 0.00$	$0.00 \times 0.477 = 0.00$	$0.33 \times 0.477 \approx 0.157$

TF-IDF & SVM Implementation

TF-IDF & SVM Implementation

```
## Importing Libraries
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

# Load the Data from CSV with specified encoding
df = pd.read_csv('spam.csv', encoding='ISO-8859-1').iloc[:,0:2]

## Check the DataFrame
print("DataFrame Head:")
df.head()
```

Output:

Out[13]:

	v1	v2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

TF-IDF & SVM Implementation

```
## Applying TF-IDF Vectorizer  
# Initialize the TF-IDF vectorizer, removing stopwords and numbers  
tfidf_vectorizer = TfidfVectorizer(stop_words='english',  
                                   token_pattern=r'\b[a-zA-Z]+\b') # Only keep alphabetic words  
  
# Fit and transform the message column (v2) to create the TF-IDF matrix  
tfidf_matrix = tfidf_vectorizer.fit_transform(df['v2'])  
  
# Convert the TF-IDF matrix to a DataFrame for better visualization  
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(),  
                        columns=tfidf_vectorizer.get_feature_names_out())  
  
# Display the TF-IDF DataFrame  
print("\nTF-IDF DataFrame:")  
tfidf_df
```

TF-IDF & SVM Implementation

Output:

Out[14]:

[illegible]

TF-IDF & SVM Implementation

```
## Displaying the feature names  
print("\nFeature names (words):")  
print(tfidf_vectorizer.get_feature_names_out())
```

```
Feature names (words):  
['aa' 'aah' 'aaniye' ... 'zoom' 'zouk'  
'zyada']
```

TF-IDF & SVM Implementation

```
## Additional Libraries
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

## Prepare Labels and Features
X=tfidf df
y=df['v1'].map({'ham': 0, 'spam': 1}) # Convert Labels to binary

## Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

## Build and Train the SVM Model
svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train, y_train)
```

TF-IDF & SVM Implementation

```
# --- Predictions on Train Set ---
y_train_pred = svm_model.predict(X_train)
print("Train Set Evaluation")
print("-----")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("\nClassification Report: \n", classification_report(y_train, y_train_pred, target_names=['ham',
'spam']))
print("\nConfusion Matrix: \n", confusion_matrix(y_train, y_train_pred))

# --- Predictions on Test Set ---
y_test_pred = svm_model.predict(X_test)

print("\nTest Set Evaluation")
print("-----")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report: \n", classification_report(y_test, y_test_pred, target_names=['ham', 'spam']))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_test_pred))
```

Q&A Session



Open floor for questions and clarifications.